

Risk Level: CRITICAL

10% safe

This code contains multiple critical security flaws, including hardcoded secrets, SQL injection, arbitrary code execution via eval, and unsafe handling of payment and user data. It should not be shipped without substantial fixes and validation around every external input path.

Fix These First

- 1 Hardcoded ``API_KEY`` and ``password`` can be leaked through source control, logs, bundles, or client-side delivery, allowing unauthorized access.
- 2 ``processPayment(amount, card)`` is vulnerable to SQL injection because ``amount`` and ``card`` are directly concatenated into ``query``.
- 3 ``parseConfig(input)`` enables arbitrary code execution if an attacker controls ``input``, because ``eval`` executes JavaScript rather than safely parsing configuration.

What the AI Assumed

- HIGH The AI assumed ``API_KEY`` can be safely hardcoded in source code and exposed wherever this file runs.
- HIGH The AI assumed ``amount`` and ``card`` in ``processPayment(amount, card)`` are trusted inputs that can be inserted into SQL with string interpolation.
- HIGH The AI assumed it is acceptable to store or log raw card data using ``console.log('Payment processed:', card)``.
- HIGH The AI assumed ``db.execute(query)`` will succeed and does not require error handling, transaction control, or parameterized queries.
- HIGH The AI assumed ``input`` passed to ``parseConfig(input)`` is trustworthy enough to evaluate as code with ``eval``.
- HIGH The AI assumed ``body`` passed to ``sendEmail(to, body)`` is safe to embed directly into HTML without escaping.
- HIGH The AI assumed hardcoding ``const password = 'admin123';`` is acceptable and that storing plaintext credentials in code is safe.

MEDIUM

The AI assumed ``userId`` in ``getUserData(userId)`` is always safe to interpolate directly into the URL path without encoding or validation.

MEDIUM

The AI assumed ``fetch('/api/users/${userId})`` always succeeds and always returns JSON.

MEDIUM

The AI assumed ``price`` and ``userAge`` in ``calculateDiscount(price, userAge)`` are valid finite numbers and that negative or nonsensical values cannot occur.

MEDIUM

The AI assumed ``to`` is always a valid recipient address and that ``mailer.send(...)`` failures do not need to be handled.

MEDIUM

The AI assumed loose equality in ``isAdmin(user)`` is acceptable and that any object passed in has a trustworthy ``role`` property.

What Could Break

HIGH

Hardcoded ``API_KEY`` and ``password`` can be leaked through source control, logs, bundles, or client-side delivery, allowing unauthorized access.

HIGH

``processPayment(amount, card)`` is vulnerable to SQL injection because ``amount`` and ``card`` are directly concatenated into ``query``.

HIGH

``parseConfig(input)`` enables arbitrary code execution if an attacker controls ``input``, because ``eval`` executes JavaScript rather than safely parsing configuration.

HIGH

``sendEmail(to, body)`` can inject unsafe HTML into outbound emails, potentially enabling phishing, malicious links, or client-side rendering abuse in email clients.

HIGH

``console.log('Payment processed:', card)`` may expose full payment card data in logs, creating PCI compliance and breach risks.

MEDIUM

``getUserData(userId)`` can fail unpredictably on network errors, non-2xx responses, or invalid JSON because there is no status or parse error handling.

MEDIUM

``calculateDiscount(price, userAge)`` may produce incorrect billing outcomes for negative values, strings, NaN, Infinity, or ages outside realistic ranges.

MEDIUM

``isAdmin(user)`` can throw if ``user`` is null/undefined, and loose equality can mask type issues in authorization logic.

Verify Checklist

- [] Verify whether this file runs client-side; if so, ``API_KEY`` and ``password`` are immediately exposed to users.
- [] Verify that ``processPayment`` never receives raw PAN/card numbers directly; if it does, remove storage and logging of ``card`` entirely and use a tokenized payment provider.
- [] Verify the database driver supports parameterized queries and replace ``db.execute(query)`` with placeholders or prepared statements.

- [] Verify every caller of `parseConfig(input)` and confirm no untrusted input can reach it; ideally remove `eval` entirely and use `JSON.parse` with schema validation.
- [] Verify `sendEmail` escapes or sanitizes `body` before rendering HTML and confirm whether plain text should be used instead.
- [] Verify `getUserData(userId)` handles non-OK responses, timeouts, and malformed JSON without crashing callers.
- [] Verify `calculateDiscount` requirements for boundary ages like 18 and 65, and validate expected behavior for decimals, negatives, and nulls.
- [] Verify authorization does not rely solely on `user.role` from untrusted objects and that `isAdmin` is not used with client-controlled data.
- [] Verify secrets are loaded from environment variables or a secret manager, never committed to source.
- [] Verify logging policy prohibits sensitive data like card numbers, API keys, passwords, and personal data.

Suggestion Cards

Hardcoded secrets

Why it matters:

Secrets in source code are commonly leaked through repositories, CI logs, bundles, and screenshots. The values `API_KEY` and `password` are immediately compromised if this file is exposed.

Fix:

Replace hardcoded credentials with environment variables or a secret manager, and fail fast when required secrets are missing.

SQL injection in `processPayment`

Why it matters:

Interpolating `amount` and `card` into `query` allows attackers to alter the SQL statement, corrupt data, or extract records.

Fix:

Use parameterized queries like `db.execute("INSERT ... VALUES (?, ?)", [amount, cardToken])` and validate input types before execution.

Raw card data handling

Why it matters:

Logging or storing `card` can violate PCI requirements and greatly increase breach impact if logs or the database are accessed.

Fix:

Do not accept raw card numbers here; accept a processor token instead, avoid logging sensitive values, and redact any incidental identifiers.

Arbitrary code execution via ``eval``

Why it matters:

Any attacker-controlled string passed to ``parseConfig`` can execute code with the application's privileges.

Fix:

Replace ``eval`` with ``JSON.parse``, then validate that the parsed result matches the expected config schema.

Unsafe HTML injection in email body

Why it matters:

Embedding unescaped ``body`` into HTML can create malicious email content or broken rendering in downstream clients.

Fix:

Escape HTML entities before insertion or send a plain-text body when rich HTML is not required.

Missing network and parse error handling

Why it matters:

In ``getUserData``, non-200 responses and invalid JSON will throw unhandled errors that may crash requests or produce misleading failures.

Fix:

Check ``response.ok``, wrap fetch and JSON parsing in try/catch, and raise descriptive errors.

Weak authorization check

Why it matters:

Using ``user.role == 'admin'`` with unchecked objects can cause authorization bugs and relies on potentially untrusted data.

Fix:

Use strict equality, handle null safely, and ensure roles come from a trusted server-side identity source.

Insufficient numeric validation in discount logic

Why it matters:

Unexpected values for ``price`` or ``userAge`` can lead to incorrect discounts, billing errors, or hidden edge-case bugs.

Fix:

Validate both inputs as finite numbers and enforce acceptable ranges before applying discount rules.